CS612 Lecture 05 — In-Class Exercises

Augmented Data Structures & Amortized Analysis · Sat 24 Oct 2026 · ungraded

A — Design the augmentation

1. Using an order-statistic tree, give an $O(\log n)$ algorithm for COUNT-RANGE(a, b), the number of stored keys in $[a, b]$ (a, b need not be stored keys).
2. Which fields can be maintained by the maintenance theorem (computable from the node and its children)? Justify each.
   - (a) the height of x's subtree
   - (b) the depth of x in the tree
   - (c) the number of red nodes in x's subtree
   - (d) the sum over the nodes v of x's subtree of the depth of v below x
   - (e) the successor key of x
3. Extend the leaderboard to report the average score of the players ranked $i..j$ in $O(\log n)$. (Sketch only; PS2 Q1 asks for the full treatment of a related query.)

B — Amortized reasoning

1. A counter supports INCREMENT, and flipping bit i costs 2^i. Is INCREMENT still $O(1)$ amortized? Give the tight amortized bound.
2. A dynamic array grows by a factor $r > 1$ ($s \to \lceil rs \rceil$). Give an accounting charge per append and prove the credit never becomes negative. What happens as $r \to 1$?
3. Logarithmic method with deletions: mark deleted items and rebuild everything when half the items are marked. Argue that DELETE is $O(\log n)$ amortized (given the item's location).

C — Choose and justify

1. An audio thread appends samples every 20 µs, and a missed deadline causes an audible click. Which growth strategy, and what kind of guarantee do you need?
2. An order-statistic tree must report the median after every update. What does that cost, and could you achieve $O(1)$ per median query?
3. An LSM-tree key-value store shows occasional 2-second write stalls. Explain them in amortized terms and suggest two mitigations.
4. True or false: "If every operation is $O(1)$ amortized, then any k consecutive operations take $O(k)$ time." Justify.

Take-home (optional)

Run `Student_Resources/dynamic_array_bench.cpp`. (1) Predict the "copies / n" column for each factor from the formula and compare. (2) Implement the de-amortized version (incremental copying) and measure the maximum single-append latency.